Microsoft Research → Autogen

↳ open source framework

↳ Powered - LLMs

↳ Agent - Co-ordinating

↓

(Conversation each other)

Society Agents

↳ Role, SM, Set of tools

↳ [Exchange messages] → Task Solved

Autogen → Autogen

Agents → Talking Agent

Exchange Messages

Planning, delegation, critique

Code generation,

Code execution,

correction

Core Components of AutoGen

The Layered Architecture

① autogen-core → runtime

② autogen-agentchat → High level, Batteries included API → Built core

Developer use

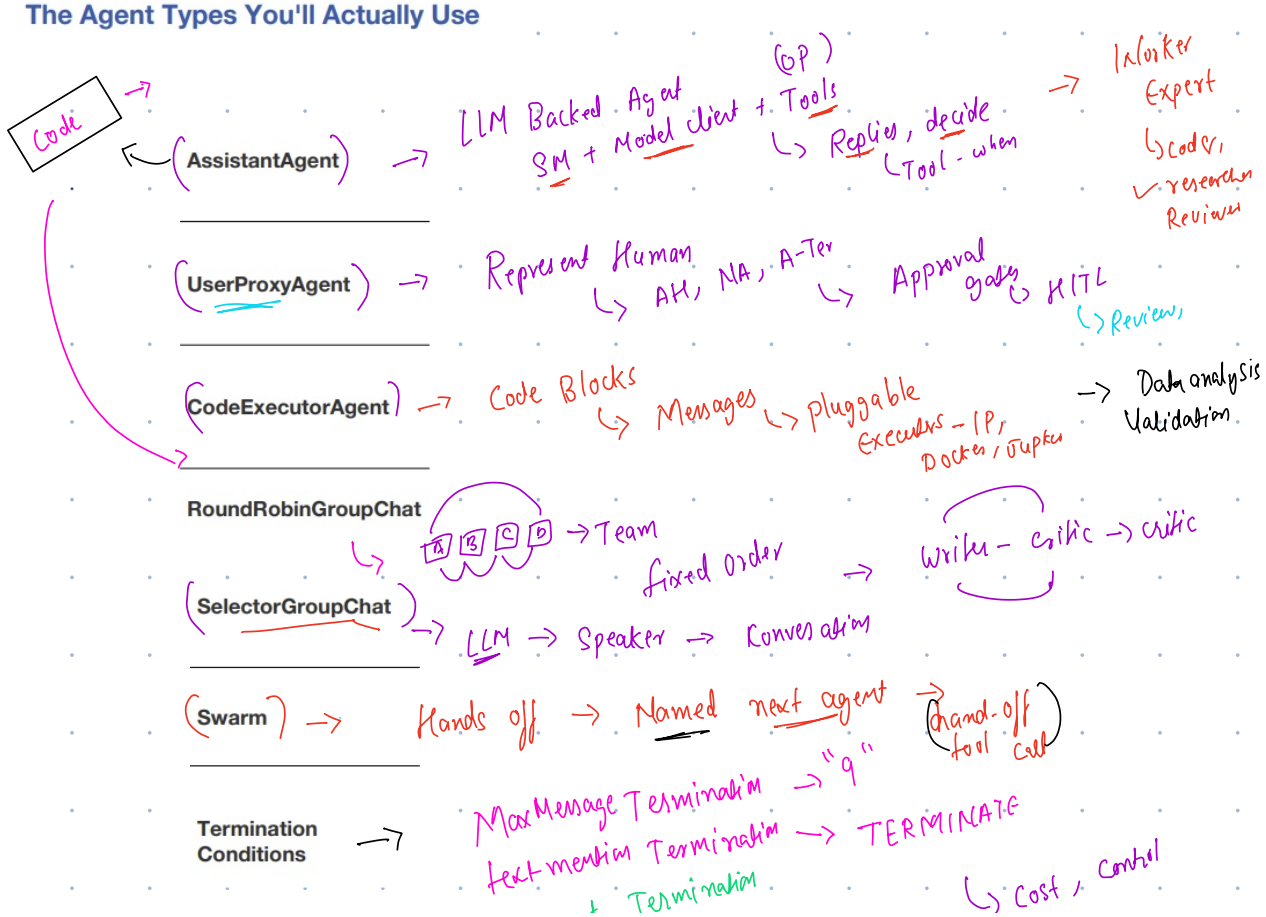
- Assistant agent, User proxy agent, Code Executor agent

- Teams
  - RoundRobinChat, SelectorGroupChat, Swarm, MagneticOneGroupChat
- Termination Conditions
  - Composable
  - Stop rule for Conversation

→ Autogen-Ext → Pluggable Extensions,
model clients - OpenAI, Azure, Anthropic
Code Executors → Tool adapters - MCP, Memory
Telemetry

→ Autogen Studio → drag-drop UI over Agent chat
→ Noncode workflow

The Agent Types You'll Actually Use



Timeow

AND / OR

Tools and Memory

↳ Tools — langchain Tool adapters, MCP - autogen-ext

Memory → Simple list Based memory Redis Store Backed

Model clients → openAI, Anthropic, local-ollama

Agentic - logic - same

swap

Approach - Building Agents

Crew AI → Crew with Roles and proces

langgraph → State Machine / Graph

Autogen → group of Actors → asynchronous messages

Workflow emerge

Done

When to Autogen

① open Ended - Exploratory Task

② coding and data alysis

③ Heavy Human In the loop + TTL

④ Multi perspective / adversarial patterns

(planner - Exc - ver,

/ debate A-A

Bad fit

→ Strict, audi table

① Confusion - Versions

② Token Cost - Scaly with Turns
(Max Message Termination

③ SandBox Code Execution

④ - auditable, Round Robin Group chat

Dimension	AutoGen	LangGraph	CrewAI	Agno
Core abstraction	Conversing agents / actor-model messaging	Explicit graph (nodes + edges) = state machine	Roles + Process (sequential/hierarchical) + Flows	Lightweight Agent objects + Teams; minimal abstraction layer
Orchestration style	Emergent from conversation, or explicit hand-off (Swarm)	Fully explicit, developer-defined graph, incl. cycles/branches	Declarative: define roles & tasks, process drives order	Direct, imperative — you call agents/teams like functions
Control over flow	Medium (dynamic selection can be non-deterministic)	High — deterministic, inspectable, resumable	Medium — process types are somewhat fixed patterns	High — thin layer, you control the loop
State management	Conversation history is the state	First-class typed state object, checkpointing built in	Task outputs passed along the process	Session/user state, storage & memory built in
Human-in-the-loop	Native (UserProxyAgent modes)	Native (interrupt/resume on the graph)	Supported, less central to the model	Supported via hooks
Code execution	Native, first-class (CodeExecutorAgent, Docker sandbox)	Possible via tools/nodes, not a core primitive	Possible via tools, not a core primitive	Possible via tools, not a core primitive
Performance / overhead	Moderate — async runtime adds some overhead	Moderate — graph engine overhead, optimized for correctness	Moderate	Very low — built for speed, small memory footprint
Learning curve	Medium–High (event-driven concepts, version split)	High (graph & state-machine thinking)	Low (most approachable)	Low–Medium (simple API, more concepts at scale)
Best known for	Research-grade multi-agent conversation & code-gen agents	Production-grade, controllable, auditable agent pipelines	Fast standup of role-based agent teams for business workflows	Raw speed, model-agnostic, production-serving many agents
Backing / ecosystem	Microsoft Research	LangChain team / ecosystem	Independent OSS, large community	Independent OSS (formerly Phidata)

- **AutoGen:** "Let the agents talk it out."
- **LangGraph:** "Draw the state machine, then run it."
- **CrewAI:** "Assign roles to a crew and define the process."
- **Agno:** "Keep it thin and fast — just agents and teams, minimal ceremony."